Data Considerations

</> Code ▾

On this page
Introduction
Rare Event Sampling
Categorical Variables and Contrasts
Convergence Problems

Introduction

There are a lot of considerations one should take with data involving a logistic regression. What happens if my event of interest is a rare occurrence in my data set? How do categorical variables influence the convergence of my logistic regression model? These are the important things we need to account for in modeling.

Rare Event Sampling

Classification models try to predict categorical target variables, typically where one of the categories is of main interest. What if this category of interest doesn't happen frequently in the data set? This is referred to as rare-event modeling and exists in many fields such as fraud, credit default, marketing responses, and rare weather events. The typical cutoff people use to decide if their event is rare is 5%. If your event occurs less than 5% of the time, you should adjust your modeling to account for this.

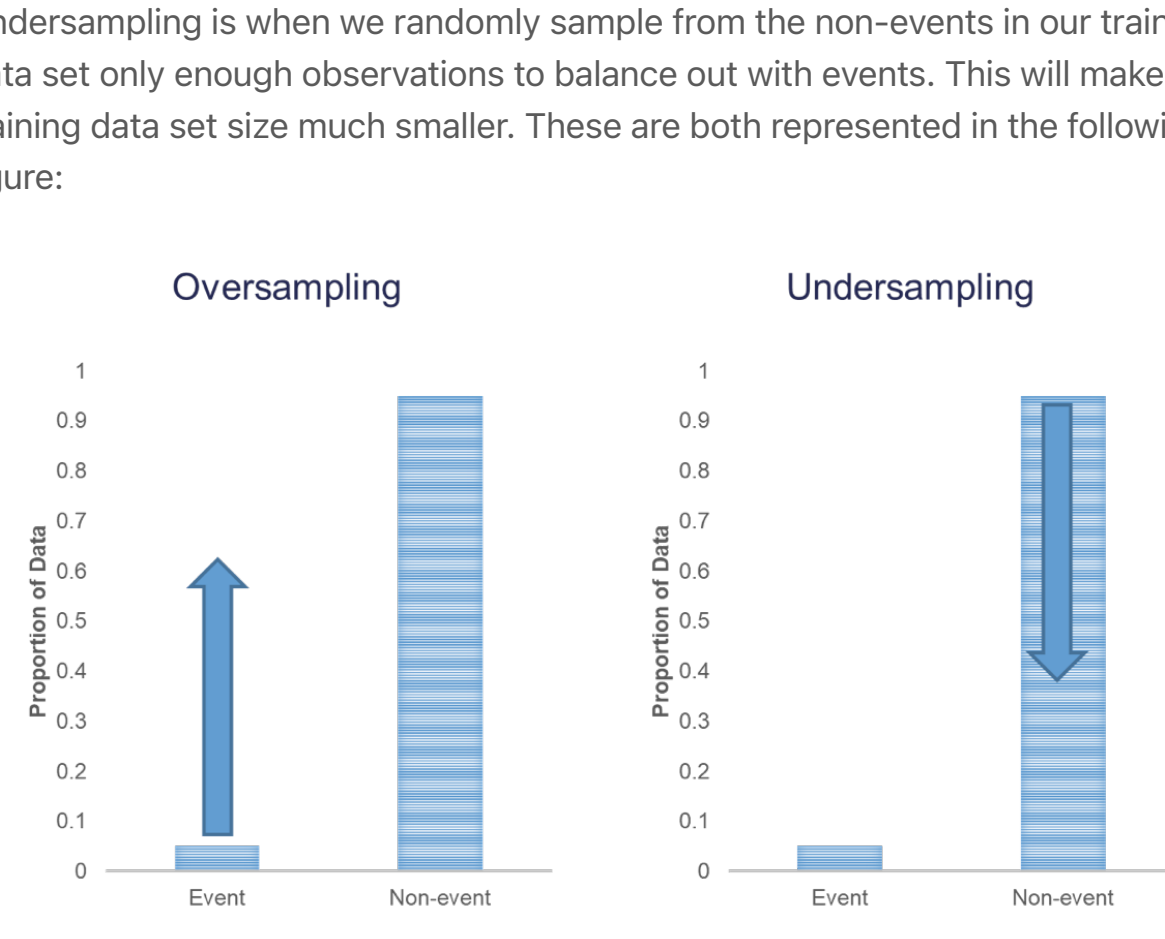
We will be using the *telecomm* data set data set to model the association between various factors and a customer churning from the company. The variables in the data set are the following:

Variable	Description
account_length	length of time with company
international_plan	yes, no
voice_mail_plan	yes, no
customer_service_calls	number of service calls
total_day_minutes	minutes used during daytime
total_day_calls	calls used during daytime
total_day_charge	cost of usage during daytime
Same as previous 3 for evening, night, international	same as above

When you have a rare event situation, you can adjust for this by balancing out the sample in your training data set. This can be done through sampling techniques typically called **oversampling**. When oversampling, you can either over or under sample your target variable to get balance.

Oversampling is when you replicate your rare event enough times to balance out with your non-events in your training data sets. There are a variety of different ways to replicate observations, but for now we will only focus on true replication - copying our actual observations at random until we get balance. This will inflate our training data set size.

Undersampling is when we randomly sample from the non-events in our training data set only enough observations to balance out with events. This will make our training data set size much smaller. These are both represented in the following figure:



Rare Event Sampling

Let's see how to perform these in each of our softwares!

R Python SAS

In R we can easily perform both over and under sampling respectively. First, we examine the proportional breakdown of our target variable with the `table` and `prop.table` functions. Specifically we want to view the churn variable. We can see below that we have roughly 5% of the customers in our data churn from the company. We also create a new *id* variable in the dataset by simply starting with the number 1 and going through the length of the dataset.

▼ Code

```
table(churn$churn)
```

```
FALSE  TRUE 
 2850   154 
```

▼ Code

```
prop.table(table(churn$churn))
```

```
     FALSE       TRUE 
0.94873502 0.05126498 
```

▼ Code

```
churn$id <- 1:length(churn$churn)
```

It is easy to perform undersampling in R with some basic data manipulation functions. Below we use the `group_by` function to group by our target variable *churn*. From the above output, we have 154 churners in our dataset. Let's sample 104 of them for the training and leave the remaining 50 for testing. Now we just need a random sample of 104 of each of the churners and non-churners from our dataset. For this we use the `sample_n` function. The training data is defined as the rows that have these row ID's and the test data set is defined as the rows that don't have these ID's. The `table` functions then show the number of observations in each data set for the *churn* variable.

▼ Code

```
library(tidyverse)

set.seed(12345)
train_u <- churn %>%
  group_by(churn) %>%
  sample_n(104)

test_u <- churn[-train_u$id,]

table(train_u$churn)
```

```
FALSE  TRUE 
  104   104 
```

▼ Code

```
table(test_u$churn)
```

```
FALSE  TRUE 
 2746    50 
```

Oversampling is also easy to perform in R. There are many ways to duplicate rows in R. First, we use the `sample_frac` function to isolate down to 70% of the observations. From there we want to isolate the churners to replicate them. For this we use the `filter` function with *churn* equal to `TRUE`. Next, we use the `slice` and `rep` functions to repeat each of the observations 10 times (set by the `each =` option). Lastly, we want to isolate the non-churners, again with the `filter` function to blend these two sets together. With the `rbind` function we combine our non-churners and the replicated churners together.

▼ Code

```
set.seed(12345)
train_o <- churn %>%
  sample_frac(0.70)

train_o_T <- train_o %>%
  filter(churn == TRUE) %>%
  slice(rep(1:n(), each = 10))

train_o_F <- train_o %>%
  filter(churn == FALSE)

train_o <- rbind(train_o_F, train_o_T)

test_o <- churn[-train_o$id,]

table(train_o$churn)
```

```
FALSE  TRUE 
 1996  1070 
```

▼ Code

```
table(test_o$churn)
```

```
FALSE  TRUE 
  854    47 
```

We now have an undersampled data set to build models on. There are two typical ways to adjust models from over or undersampling - adjusting the intercept and weighting.

Let's examine this data set and how we can adjust our models accordingly with each technique.

Adjusting the Intercept

One of the ways of adjusting your logistic regression model to account for the sampling to balance the events with non-events is by adjusting the intercept from the logistic regression model. The intercept is what sets the "baseline" probability in your model which is now too high. You can adjust your predicted probabilities from the regression model to account for this incorrect intercept. They can be adjusted with the following equation:

$$\hat{p}_i = \frac{\hat{p}_i^* \rho_0 \pi_1}{(1-\hat{p}_i^*)\rho_1 \pi_0 + \hat{p}_i^* \rho_0 \pi_1}$$

where $\hat{p}_i^*$ is the unadjusted predicted probability, π_1 and π_0 are the population proportions of your target variable categories (1 and 0), and ρ_1 and ρ_0 are the sample proportions of your target variable categories (1 and 0).

Let's see how to do this in each of our softwares!

R Python SAS

Luckily, this is rather easy to do in R as we can directly adjust the predicted probability vector with this equation. First, we build a logistic regression model as seen in the previous section using the `glm` function on our undersampled training data. We can use the `predict` function to obtain our predicted probabilities from this model. These are biased probabilities however because of our sampling. By simply applying the equation above on the predicted probability vector we can get unbiased predicted probabilities.

▼ Code

```
logit.model <- glm(churn ~ factor(international_plan) +
                     factor(voice_mail_plan) +
                     total_day_charge +
                     customer_service_calls,
                   data = train_u, family = binomial(link = "lo
summary(logit.model)
```

```
Call:
glm(formula = churn ~ factor(international_plan) + factor(voice_
    total_day_charge + customer_service_calls, family = binomial
    data = train_u)

Coefficients:
                              Estimate Std. Error z value Pr(>|z
(Intercept)                   -5.81880    0.95939  -6.065 1.32e-
factor(international_plan)yes  2.97995    0.57057   5.223 1.76e-
factor(voice_mail_plan)yes    -0.85107    0.41372  -2.057  0.03
total_day_charge               0.12898    0.02234   5.773 7.79e-
customer_service_calls         0.78520    0.14947   5.253 1.50e-
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 288.35  on 207  degrees of freedom
Residual deviance: 195.24  on 203  degrees of freedom
AIC: 205.24

Number of Fisher Scoring iterations: 5
```

▼ Code

```
test_u_p_bias <- predict(logit.model, newdata = test_u, type =
test_u_p <- (test_u_p_bias*(104/208)*(154/3004))/((1-test_u_p_b

test_u <- data.frame(test_u, 'Pred' = test_u_p)

head(test_u_p)
```

```
         1          2          3          4          5 
0.04788873 0.00516951 0.03230802 0.91516214 0.56312205 0.2976687
```

We then store these as a new column in our test data set.

Weighted Observations

The other technique for adjusting the model for over or under sampling is by building the model with weighted observations so the adjustment happens while building instead of after the fact. In weighting the observations we use weighted MLE instead of the typical MLE so each observation doesn't have the same weight in the estimation of the parameters for our model. The only question now is what are the weights. We derive the weights from the same formula we had for the intercept adjustment. The weight for the rare event is 1, while the weight for the non-event is $\rho_1\pi_0/\rho_0\pi_1$. For our data set this makes the weights 1 and 18.51 respectively.

Let's see this approach in each of our softwares!

R Python SAS

This is easily done in R by creating a new *weight* variable in our training data with the `ifelse` function. If our *churn* variable takes a value of *TRUE* then that observation has a weight of 1, while the other observations have a weight of 18.51. From there we build the logistic regression model as we learned in the previous section using the `glm` function. The only new option is the `weights =` option where you specify the vector of weights for the training data set so weighted MLE can be performed.

▼ Code

```
train_u$weight <- ifelse(train_u$churn == 'TRUE', 1, 18.51)

logit.model.w <- glm(churn ~ factor(international_plan) +
                       factor(voice_mail_plan) +
                       total_day_charge +
                       customer_service_calls,
                     data = train_u, family = binomial(link = "
                     weights = weight)
summary(logit.model.w)
```

```
Call:
glm(formula = churn ~ factor(international_plan) + factor(voice_
    total_day_charge + customer_service_calls, family = binomial
    data = train_u, weights = weight)

Coefficients:
                              Estimate Std. Error z value Pr(>|z
(Intercept)                   -9.76986    0.69645 -14.028  < 2e-
factor(international_plan)yes  3.33574    0.32426  10.287  < 2e-
factor(voice_mail_plan)yes    -1.07459    0.27105  -3.965 7.35e-
total_day_charge               0.16322    0.01647   9.912  < 2e-
customer_service_calls         0.72694    0.08809   8.252  < 2e-
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 820.53  on 207  degrees of freedom
Residual deviance: 585.50  on 203  degrees of freedom
AIC: 599.9

Number of Fisher Scoring iterations: 6
```

▼ Code

```
test_u_p_w <- predict(logit.model.w, newdata = test_u, type = "

test_u <- data.frame(test_u, 'Pred_w' = test_u_p_w)

head(test_u_p_w)
```

```
          1           2           3           4           5 
0.059450403 0.003561721 0.046710480 0.965360041 0.592005413 0.44
```

Now we can analyze our results above knowing that the predictions and model have already been adjusted for our sampling because of the weighted approach. We can use the `predict` function to obtain the predictions from our model and make them a new vector in our validation data set.

Which approach is better? The more common approach is the weighted observation approach. Empirical simulation studies have shown that for large sample sizes (n > 1000), the weighted approach is better. In small sample sizes (n < 1000), the adjusted intercept approach is only better when you correctly specify the model. In other words, if you plan on doing variable selection because you are unsure if you have the correct variables in your model, then your model may be misspecified in its current form until after your perform variable selection. That being the case, it is probably safer to use the weighted approach. This is why most people use the weighted approach.

Categorical Variables and Contrasts

Categorical variables can provide great value to any model. Specifically, we might be interested in the differences that might exist across categories in a logistic regression model with regards to our target variable. However, by default, logistic regression models only provide certain categorical comparisons based on the coding of our categorical variables. Through contrasts we can compare any two categories or combination of categories that we desire.

Let's see how to do this in each of our softwares!

R Python SAS

R and the `glht` function can easily test all combinations of categorical variable categories for inference on the target variable. First we need the variables in a workable format. We need to make the categorical variables factors outside the `glm` function. This new factor representation of *customer_service_calls* is represented by the *fcsc* variable using the `factor` function. Then we fit our model as usual with the `glm` function. With the `glht` function, we test each combination of the levels of the variable for statistical differences. The `linfct` option specifies the linear function we are trying to test. This option can be used to test all combinations as we do below or individual combinations (like the TEST statement in SAS, but are not shown here). With the `mcp` function we specify that we want multiple comparisons with the `Tukey` adjustment on the *fcsc* variable.

▼ Code

```
train_u$fcsc <- factor(train_u$customer_service_calls)

logit.model.w.2 <- glm(churn ~ factor(international_plan) +
                         factor(voice_mail_plan) +
                         total_day_charge +
                         fcsc,
                       data = train_u, family = binomial(link =
                       weights = weight)
summary(logit.model.w.2)
```

```
Call:
glm(formula = churn ~ factor(international_plan) + factor(voice_
    total_day_charge + fcsc, family = binomial(link = "logit"),
    data = train_u, weights = weight)

Coefficients:
                               Estimate Std. Error z value Pr(>
(Intercept)                   -10.14898    0.81614 -12.435  < 2
factor(international_plan)yes   3.29320    0.34765   9.473  < 2
factor(voice_mail_plan)yes      -0.99871    0.30330  -3.293 0.00
total_day_charge                 0.17916    0.01788  10.020  < 2
fcsc1                            0.49930    0.36184   1.380 0.16
fcsc2                            1.44557    0.40012   3.613 0.00
fcsc3                            1.22910    0.44653   2.753 0.00
fcsc4                            3.61477    0.51999   6.952 3.61
fcsc5                            2.38244    0.69874   3.410 0.00
fcsc6                           22.86252  799.55896   0.029 0.97
fcsc7                           21.52806 1028.81098   0.021 0.98
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 820.53  on 207  degrees of freedom
Residual deviance: 545.74  on 197  degrees of freedom
AIC: 571.92

Number of Fisher Scoring iterations: 14
```

▼ Code

```
library(multcomp)

summary(glht(logit.model.w.2, linfct = mcp(fcsc = "Tukey")))
```

```

	 Simultaneous Tests for General Linear Hypotheses

Multiple Comparisons of Means: Tukey Contrasts


Fit: glm(formula = churn ~ factor(international_plan) + factor(v
    total_day_charge + fcsc, family = binomial(link = "logit"),
    data = train_u, weights = weight)

Linear Hypotheses:
           Estimate Std. Error z value Pr(>|z|)    
1 - 0 == 0   0.4993     0.3618   1.380  0.81745    
2 - 0 == 0   1.4456     0.4001   3.613  0.00461 ** 
3 - 0 == 0   1.2291     0.4465   2.753  0.07439 .  
4 - 0 == 0   3.6148     0.5200   6.952  < 0.001 ***
5 - 0 == 0   2.3824     0.6987   3.410  0.00958 ** 
6 - 0 == 0  22.8625   799.5590   0.029  1.00000    
7 - 0 == 0  21.5281  1028.8110   0.021  1.00000    
2 - 1 == 0   0.9463     0.3627   2.609  0.10896    
3 - 1 == 0   0.7298     0.4043   1.805  0.53232    
4 - 1 == 0   3.1155     0.4811   6.476  < 0.001 ***
5 - 1 == 0   1.8831     0.6683   2.818  0.06231 .  
6 - 1 == 0  22.3632   799.5589   0.028  1.00000    
7 - 1 == 0  21.0288  1028.8109   0.020  1.00000    
3 - 2 == 0  -0.2165     0.4001  -0.541  0.99906    
4 - 2 == 0   2.1692     0.4649   4.666  < 0.001 ***
5 - 2 == 0   0.9369     0.7019   1.335  0.84161    
6 - 2 == 0  21.4170   799.5589   0.027  1.00000    
7 - 2 == 0  20.0825  1028.8109   0.020  1.00000    
4 - 3 == 0   2.3857     0.4942   4.828  < 0.001 ***
5 - 3 == 0   1.1533     0.7266   1.587  0.68743    
6 - 3 == 0  21.6334   799.5589   0.027  1.00000    
7 - 3 == 0  20.2990  1028.8109   0.020  1.00000    
5 - 4 == 0  -1.2323     0.7743  -1.592  0.68452    
6 - 4 == 0  19.2478   799.5589   0.024  1.00000    
7 - 4 == 0  17.9133  1028.8110   0.017  1.00000    
6 - 5 == 0  20.4801   799.5591   0.026  1.00000    
7 - 5 == 0  19.1456  1028.8111   0.019  1.00000    
7 - 6 == 0  -1.3345  1302.9759  -0.001  1.00000    
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
(Adjusted p values reported -- single-step method)
```

In the table above you we have the statistical tests of 28 combinations of the *customer_service_calls* variable levels. These tests tell us if there is a statistical difference between the two categories with respect to customers churning. You will notice some rather interesting odds ratios and confidence intervals for a couple of the tests. We will talk further about these issues in the convergence section below.

Convergence Problems

One of the downsides of maximum likelihood estimation is that there is no closed form solution in logistic regression. This means that an algorithm must converge and find the point that maximizes our logistic regression likelihood instead of just calculating a known answer like OLS in linear regression. This means that the algorithm might not converge.

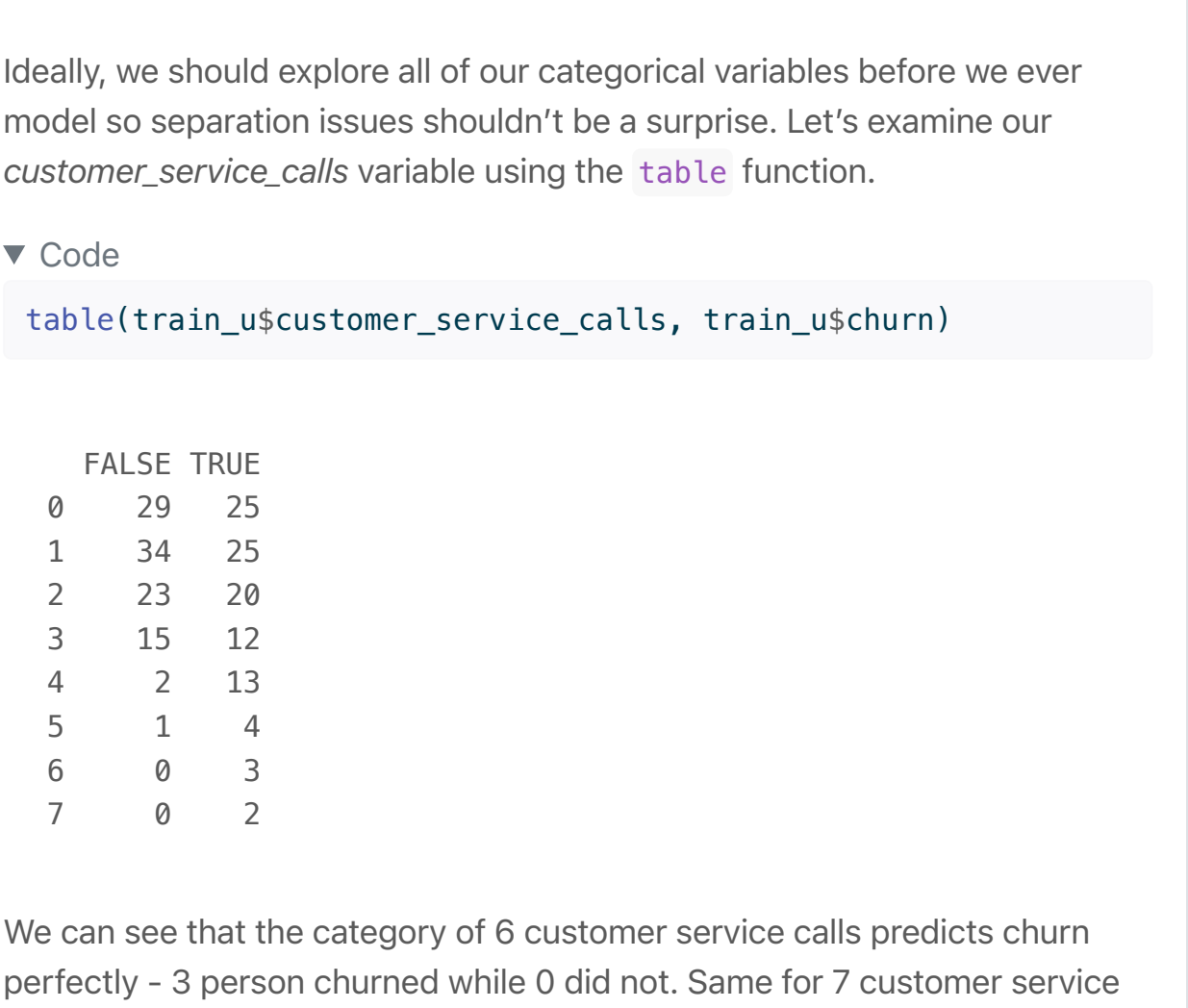
Categorical variables are the typical culprit for causing problems in convergence (however, rarely continuous variables can do this as well). This lack of convergence from categorical variables is from *linear separation* or *quasi-complete separation*. Complete linear separation occurs when some combination of the predictors perfectly predict **every** outcome as you can see in the table below:

Variable	Yes	No	Logit
Group A	100	0	Infinity
Group B	0	50	Negative Infinity

Quasi-complete separation occurs when the outcome can be perfectly predicted for only a subset of the data as shown in the table below:

Variable	Yes	No	Logit
Group A	77	23	1.39
Group B	0	50	Negative Infinity

The reason this poses a problem is that the likelihood function doesn't actually have a maximum.



Remember that the logistic function is completely bounded within 0 and 1. Since these categories perfectly predict the target variable, we need a prediction of 0 or 1 for the probability which cannot be obtained in the logistic regression without infinitely positive (or negative) parameter estimates.

Let's see if this is a problem in our data set (hint... it is!) and how to address this problem in our software!

R Python SAS

Ideally, we should explore all of our categorical variables before we ever model so separation issues shouldn't be a surprise. Let's examine our *customer_service_calls* variable using the `table` function.

▼ Code

```
table(train_u$customer_service_calls, train_u$churn)
```

```
   
    FALSE TRUE
  0    29   25
  1    34   25
  2    23   20
  3    15   12
  4     2   13
  5     1    4
  6     0    3
  7     0    2
```

We can see that the category of 6 customer service calls predicts churn perfectly - 3 person churned while 0 did not. Same for 7 customer service calls. Every other category has at least one person who churned and one who didn't churn. So this is an example of quasi-complete separation.

In case you don't explore your data ahead of time, you will notice interesting results in categories with separation issues. Let's examine the parameter estimates for 6 and 7 customer service calls.

▼ Code

```
logit.model.w <- glm(churn ~ factor(international_plan) +
                       factor(voice_mail_plan) +
                       total_day_charge +
                       factor(customer_service_calls),
                     data = train_u, family = binomial(link = "
                     weights = weight)
summary(logit.model.w)
```

```
Call:
glm(formula = churn ~ factor(international_plan) + factor(voice_
    total_day_charge + factor(customer_service_calls), family =
    data = train_u, weights = weight)

Coefficients:
                                 Estimate Std. Error z value Pr
(Intercept)                     -10.14898    0.81614 -12.435  <
factor(international_plan)yes     3.29320    0.34765   9.473  <
factor(voice_mail_plan)yes       -0.99871    0.30330  -3.293 0.
total_day_charge                  0.17916    0.01788  10.020  <
factor(customer_service_calls)1   0.49930    0.36184   1.380 0.
factor(customer_service_calls)2   1.44557    0.40012   3.613 0.
factor(customer_service_calls)3   1.22910    0.44653   2.753 0.
factor(customer_service_calls)4   3.61477    0.51999   6.952 3.
factor(customer_service_calls)5   2.38244    0.69874   3.410 0.
factor(customer_service_calls)6  22.86252  799.55896   0.029 0.
factor(customer_service_calls)7  21.52806 1028.81098   0.021 0.
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 820.53  on 207  degrees of freedom
Residual deviance: 545.74  on 197  degrees of freedom
AIC: 571.92

Number of Fisher Scoring iterations: 14
```

Notice the category of 6 and 7 compared to the rest. Their parameter estimates are rather high (remember that odds ratios are exponentiated parameter estimates so 20+ is REALLY large). These are definite signs of convergence problems, especially considering the p-value for these is close to 1!

Once possible solution to convergence problems in logistic regression is to perform exact logistic regression. This method is computationally more expensive and should really only be used if you need an estimate for the specific category that is causing the separation issues. The `logistf` package in R can perform this analysis but is not shown here.

A more common solution to separation concerns is to transform the categorical variable. Instead of having a category with separation issues, you can combine this category with another category to remove the separation concerns. Below we are creating a "4+" category that contains the categories of 4, 5, 6, and 7. This will remove the separation issues as you can see as we rerun the logistic regression.

▼ Code

```
train_u$customer_service_calls_c <- as.character(train_u$custom
train_u$customer_service_calls_c[which(train_u$customer_service

table(train_u$customer_service_calls_c, train_u$churn)
```

```
     FALSE TRUE
  0     29   25
  1     34   25
  2     23   20
  3     15   12
  4+     3   22
```

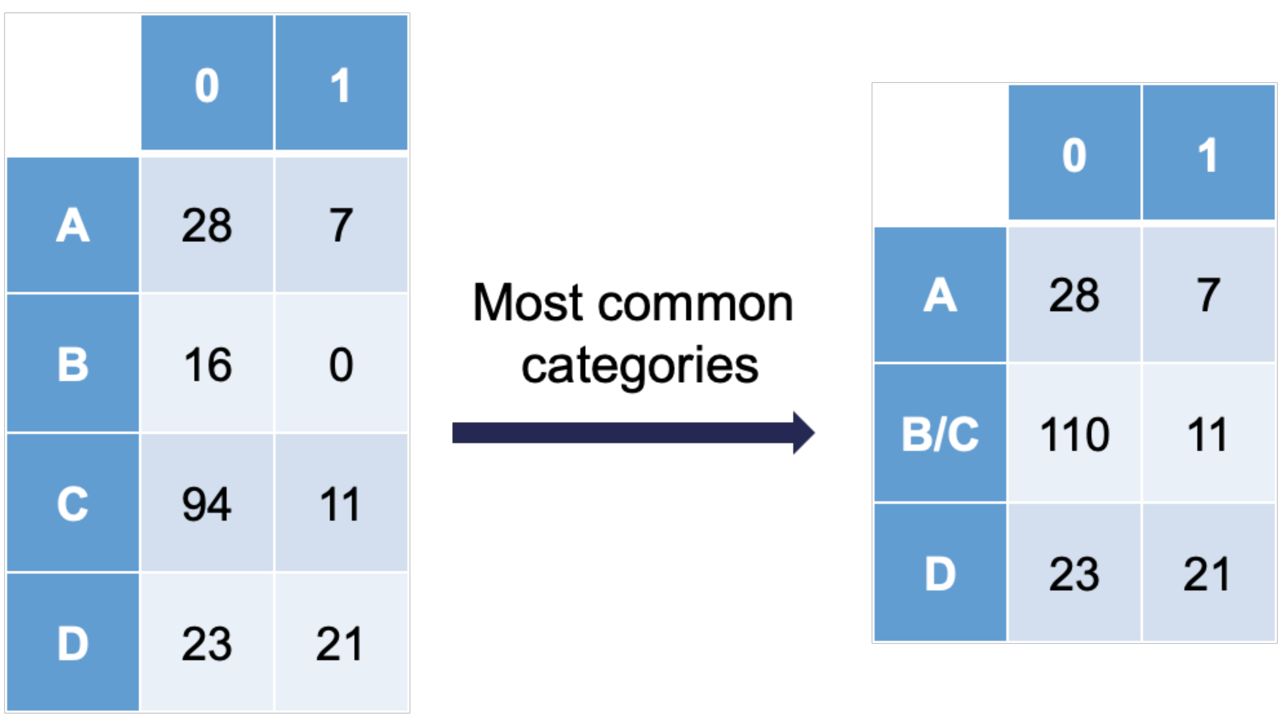
▼ Code

```
logit.model.w <- glm(churn ~ factor(international_plan) +
                             factor(voice_mail_plan) +
                             total_day_charge +
                             factor(customer_service_calls_c),
                     data = train_u, family = binomial(link = "
                     weights = weight)
summary(logit.model.w)
```

```

Call:
glm(formula = churn ~ factor(international_plan) + factor(voice_
    total_day_charge + factor(customer_service_calls_c), family
    data = train_u, weights = weight)

Coefficients:
                                   Estimate Std. Error z value P
(Intercept)                        -9.18049    0.73431 -12.502
factor(international_plan)yes       3.08132    0.33255   9.266
factor(voice_mail_plan)yes         -1.14286    0.27819  -4.108 3
total_day_charge                    0.15726    0.01647   9.551
factor(customer_service_calls_c)1   0.44991    0.35419   1.270
factor(customer_service_calls_c)2   1.25333    0.38493   3.256
factor(customer_service_calls_c)3   1.03757    0.43343   2.394
factor(customer_service_calls_c)4+  3.45947    0.42510   8.138 4
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 820.53  on 207  degrees of freedom
Residual deviance: 580.00  on 200  degrees of freedom
AIC: 600.53

Number of Fisher Scoring iterations: 6
```

Notice how we no longer have the large parameter estimates since we have corrected our converge problems.

Ordinal variables are easy to combine categories because you can just combine separation categories with categories on either side. However, with nominal variables we typically combine the separation causing category with the category that has the relationship with the target variable that is the most similar. The following plot is an example of this:



	0	1
A	28	7
B	16	0
C	94	11
D	23	21

Most common categories

	0	1
A	28	7
B/C	110	11
D	23	21

Notice how category C is most similar to category B (the problem category) in terms of its ratio of 0′s and 1′s. Formally, this method of combining categories is also called the Greenacre method.

← Binary Logistic Regression

Subset Selection and Diagnostics →